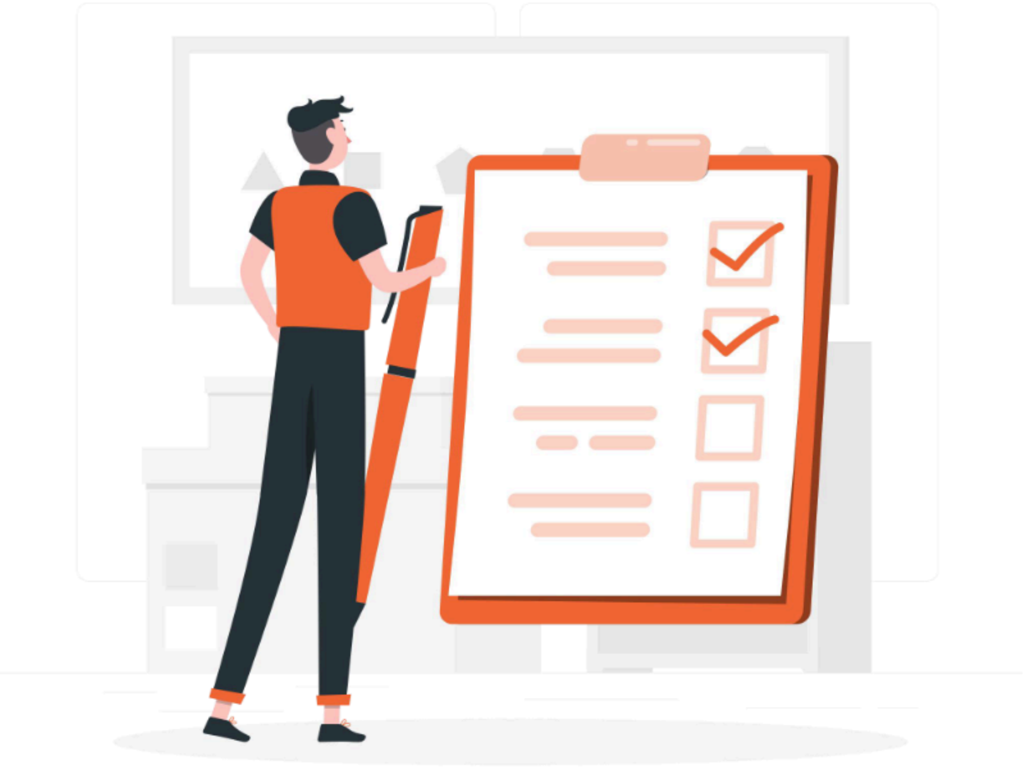
LLM API Integration & Architecture

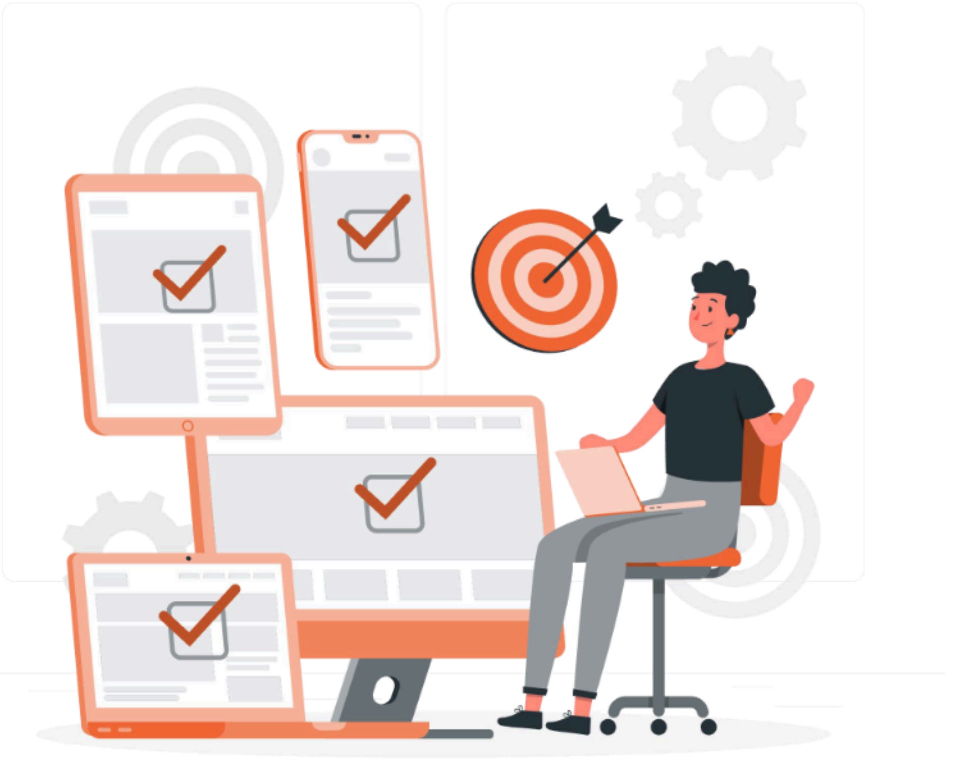
Lesson

1. Calling Azure OpenAI API
2. Embedding generation
3. Token counting & cost estimation
4. Processing large context inputs
5. Vector database workflow



Learning Outcome

1. Use Azure OpenAI API for generation and embeddings.
2. Manage tokens, costs, and large-context inputs efficiently.
3. Apply vector database workflows for retrieval and AI applications.



HACKTIV8

From Prompt to Product

Production Changes the Problem

A frontend can collect intent, but the backend is where routing, secrets, logging, fallback logic, and response shaping belong.

Client Application
Captures user intent and displays results

→

Backend API
Protects secrets chooses deployments logs usage

→

Azure OpenAI
Runs generation and embedding models

→

Optional Retrieval
Supplies fresh or private context first

Control

The backend decides deployment, instructions, and exposed response fields.

Observability

Routes can record tokens, latency, failure patterns, and request volume.

Extensibility

Retrieval, caching, and orchestration can be added without rewriting the client.

HACKTIV8

Azure Integration

Setup

Configuration

Endpoint
Authentication
Chat deployment
Embedding deployment

Control

Backend route

Loads env vars builds OpenAI client selects the right deployment

Call

responses.create(...)

Sends input to the Azure base_url using the chosen deployment name

Observe

Application response

Returns application output plus usage fields for monitoring

Key operational reminder

In Azure, the app usually calls the deployment name. That is the application-facing identifier the backend needs to know.

Authentication choices

API keys are common in demos. Microsoft Entra ID matters when teams want keyless, centrally managed authentication.

Why the backend owns it

The client should not know secrets or deployment routing. Keeping those decisions server-side preserves flexibility.

Calling Azure OpenAI API

HACKTIV8

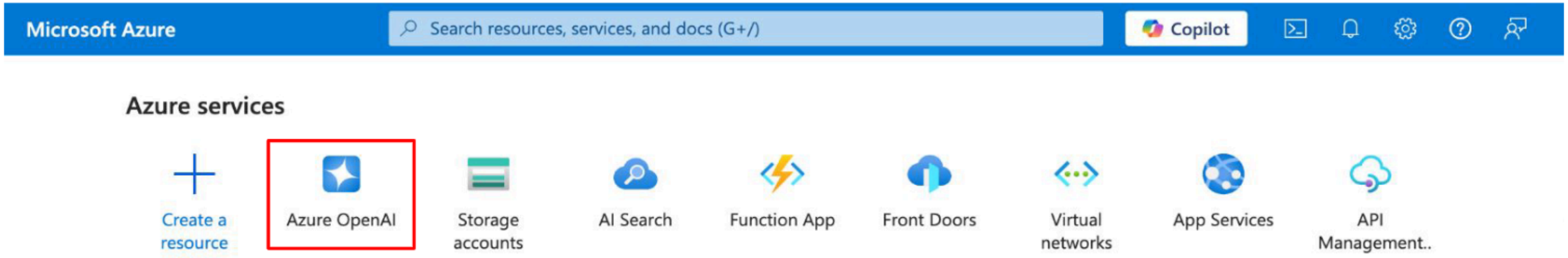
Azure OpenAI: Overview

Azure OpenAI is a cloud service from Microsoft that enables the use of advanced AI models such as those from OpenAI, hosted securely on the Microsoft Azure platform.

Feature	OpenAI	Azure OpenAI
Infrastructure	Managed by OpenAI independently	Managed by Microsoft (Azure)
Data Privacy	Customer data used for training	No customer data used for training
Compliance	OpenAI Enterprise standard	Full Azure compliance (HIPAA, SOC2)
Support/SLA	Platform support	Enterprise-grade SLAs
Security	Standard API security	VNETs, RBAC, Private Links

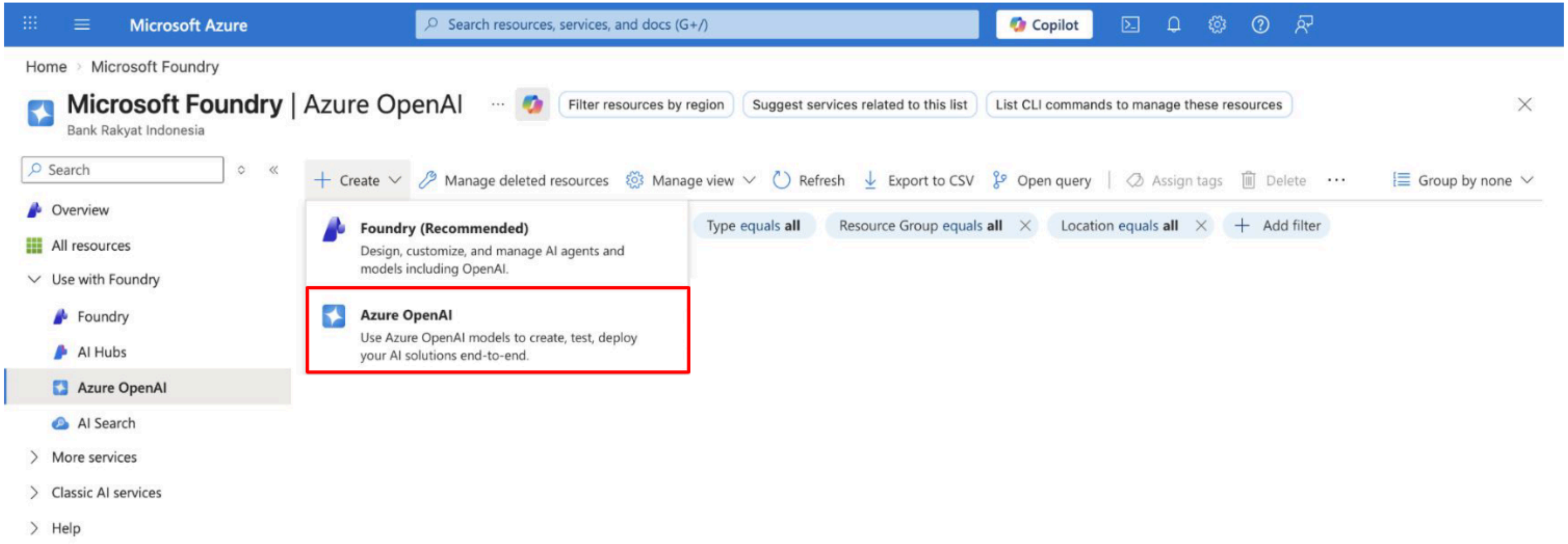
Azure OpenAI: Create Instance

- Visit URL https://portal.azure.com/
- Login with your email and password
- It will open the azure portal
- Click **Azure OpenAI**



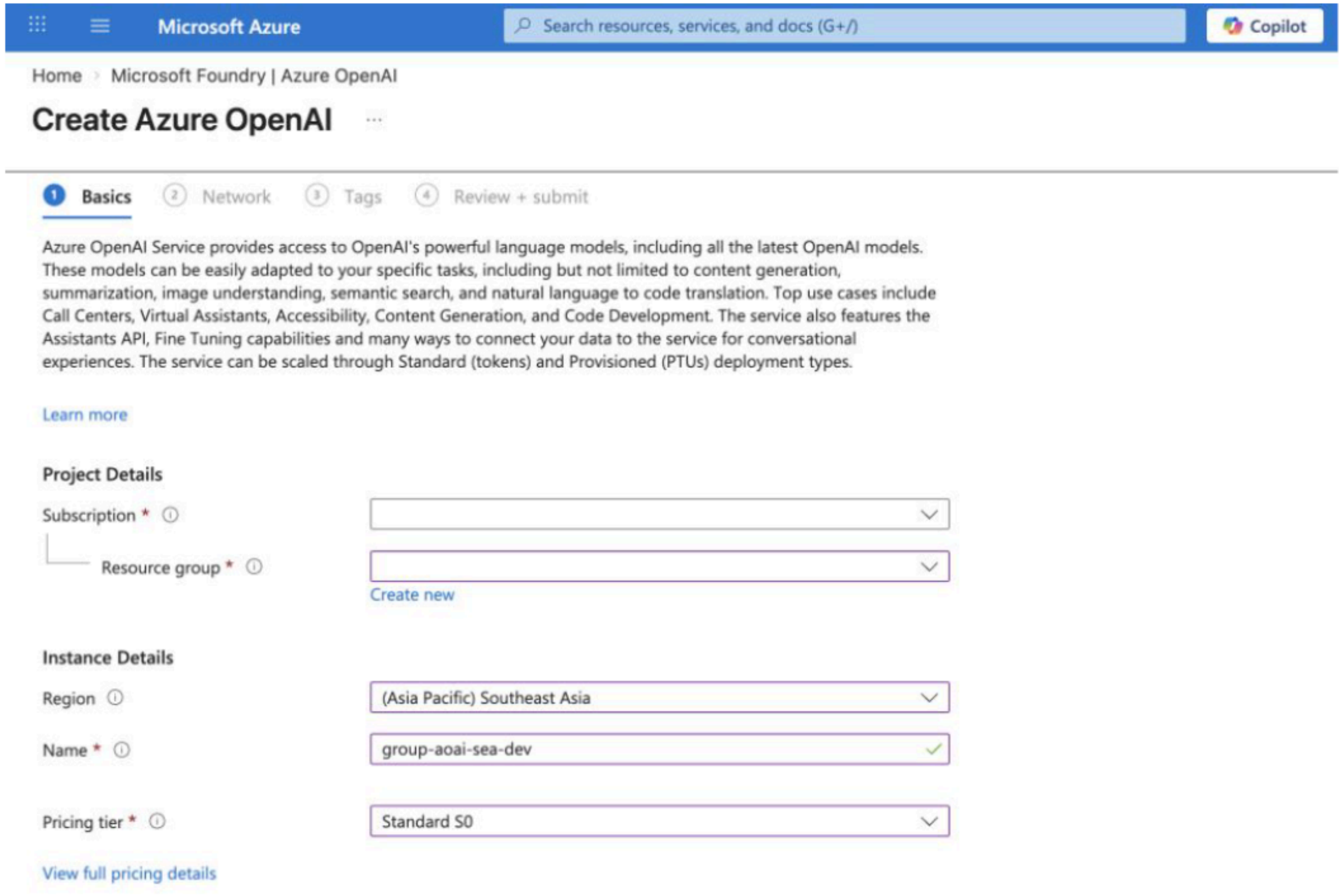
Azure OpenAI: Create Instance

- It will open the Microsoft Foundry or Azure OpenAI page.
- To create instance, click **Create.**
- Then choose **Azure OpenAI.**



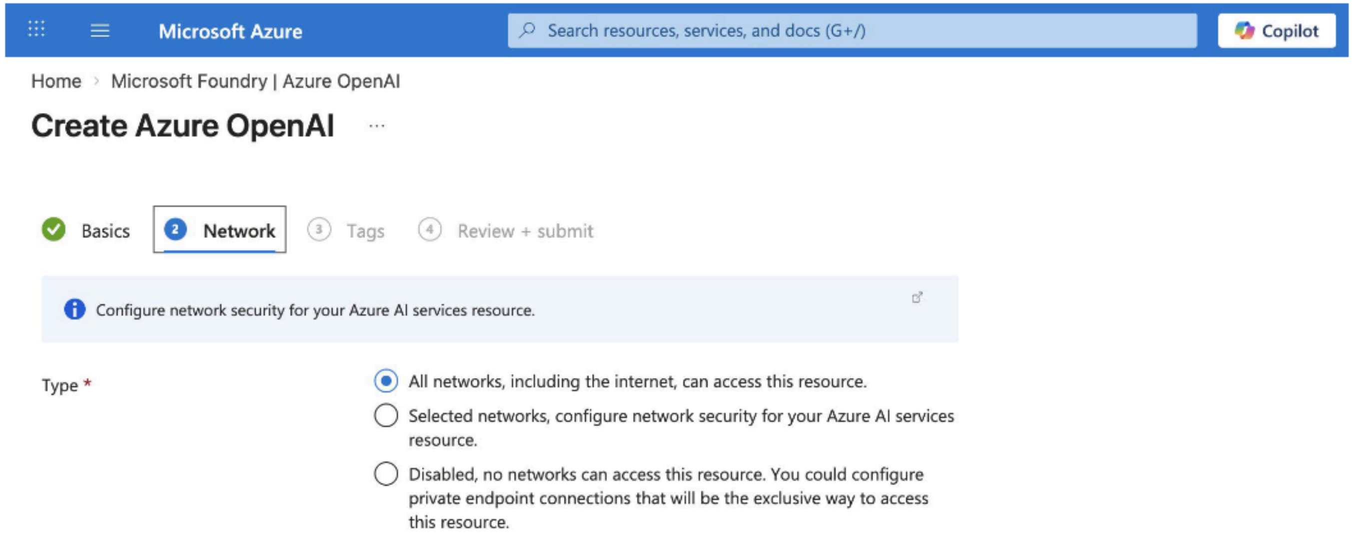
Azure OpenAI: Create Instance

- Fill the project details in basics section.
- Choose your subscription, resource group, and region based on your company's policy.
- For the name, use lowercase letter with format: {group}-{instance}-{region}-{environment}.
  For example: bsi-aoai-sea-dev
- Choose standard pricing tier
- Then, click **Next.**



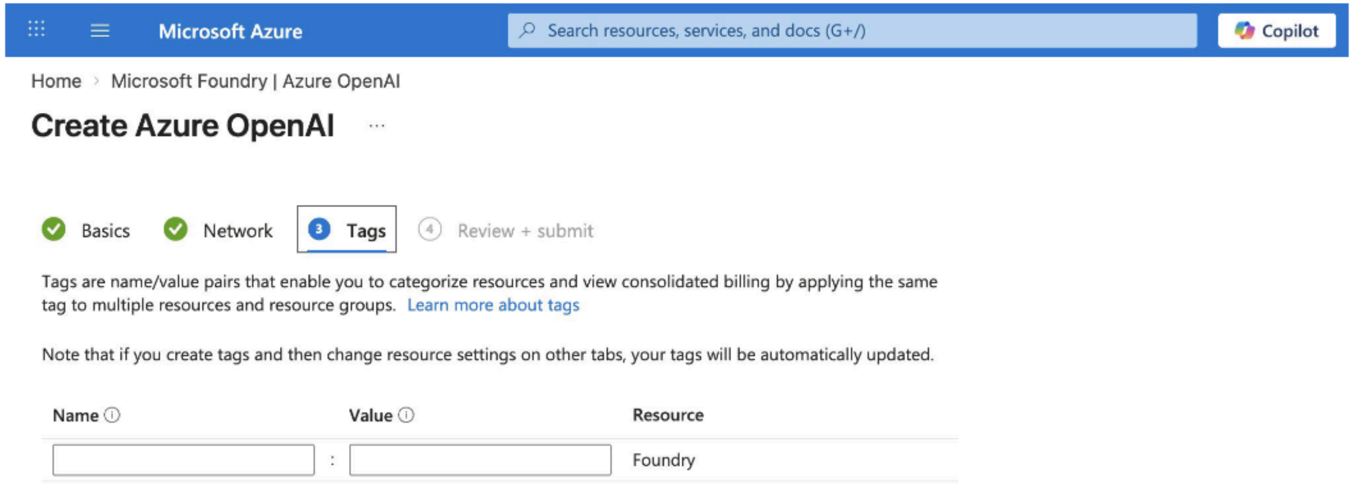
Azure OpenAI: Create Instance

- Configure the network security in network section.
- Choose all networks if there are no customize network security for your service.
- Then, click **Next.**



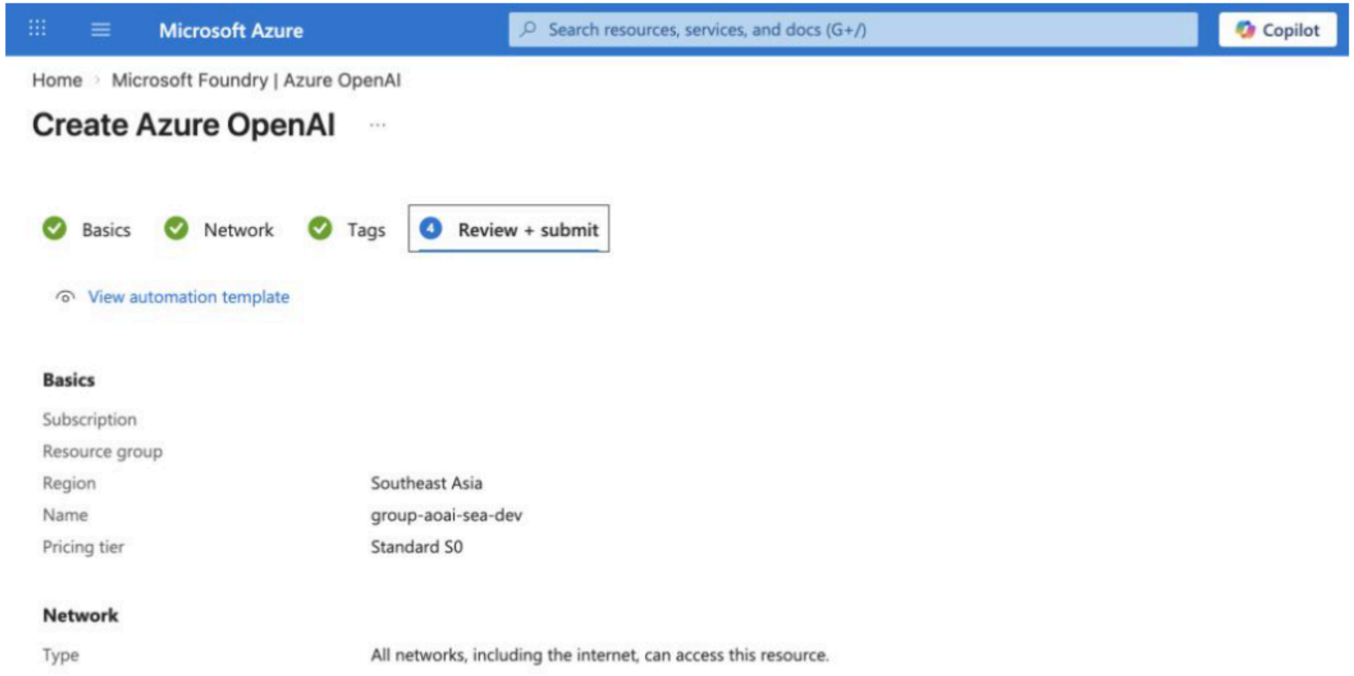
Azure OpenAI: Create Instance

- Configure the name/value in tags section.
- Tags are name/value pairs that enable you to categorize resource.
- Then, click **Next.**



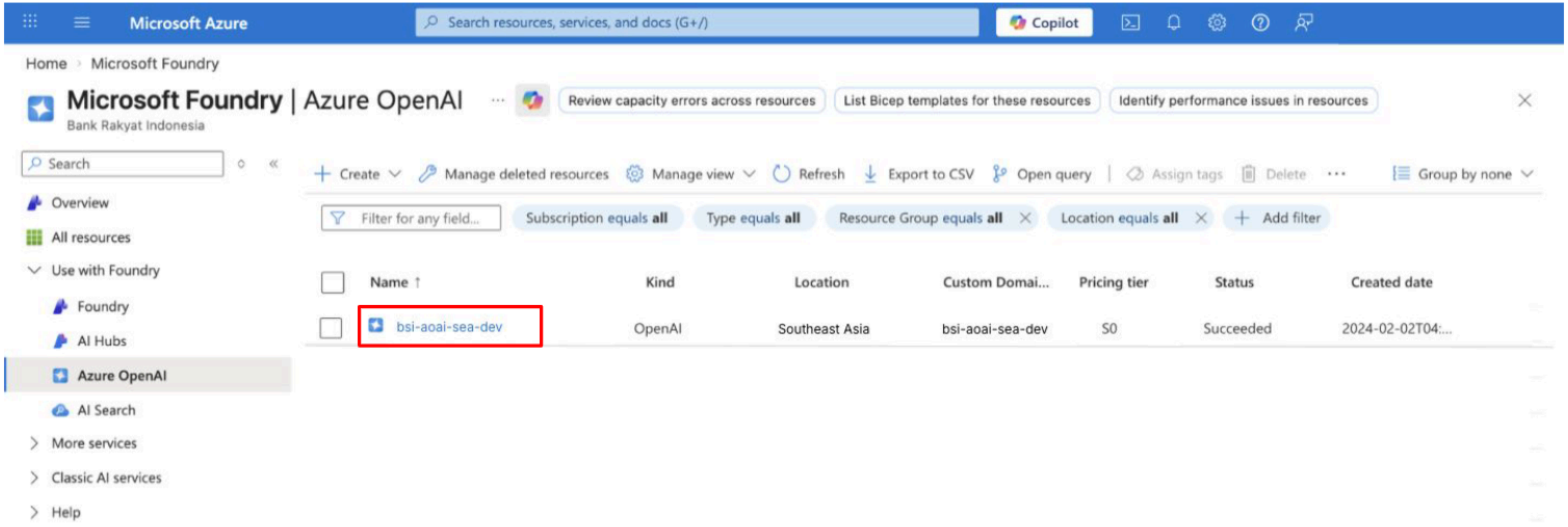
Azure OpenAI: Create Instance

- Review the basics and network settings that have already been configured before submitting.
- If everything is correct, click **Submit**.



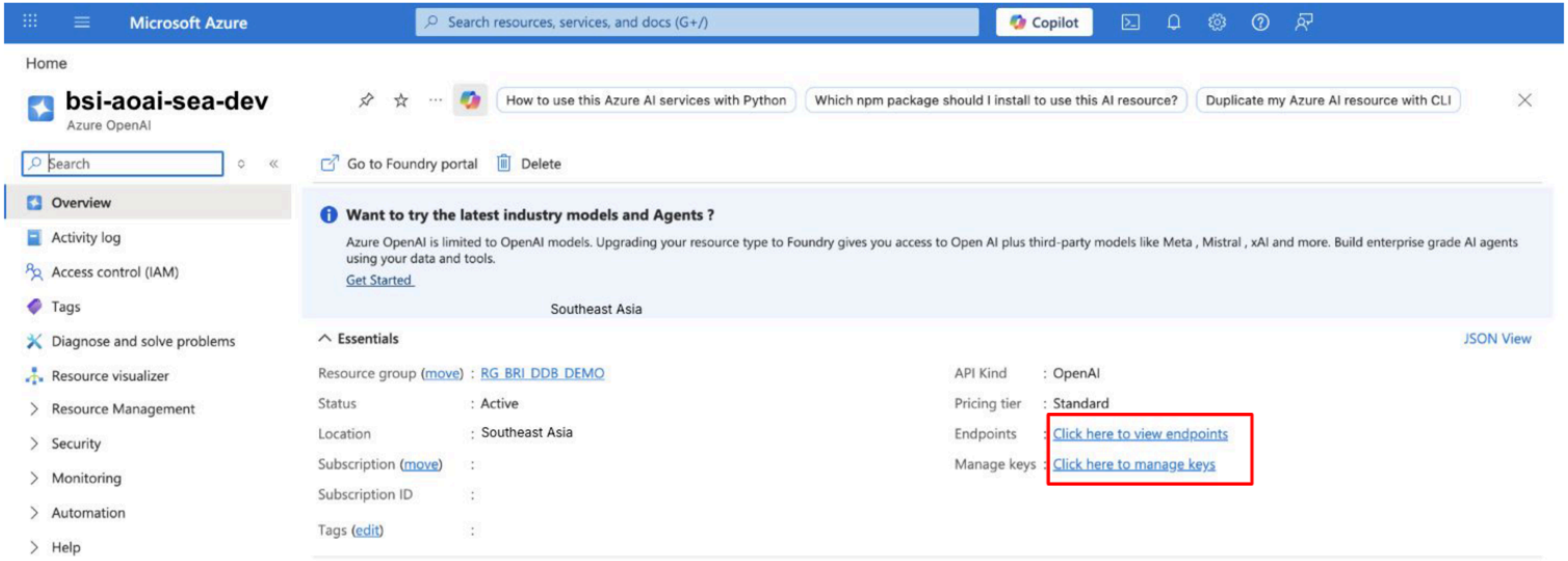
Azure OpenAI: Get the API Key

- Open the Microsoft Foundry or Azure OpenAI page again.
- Then, click the instance that was created.



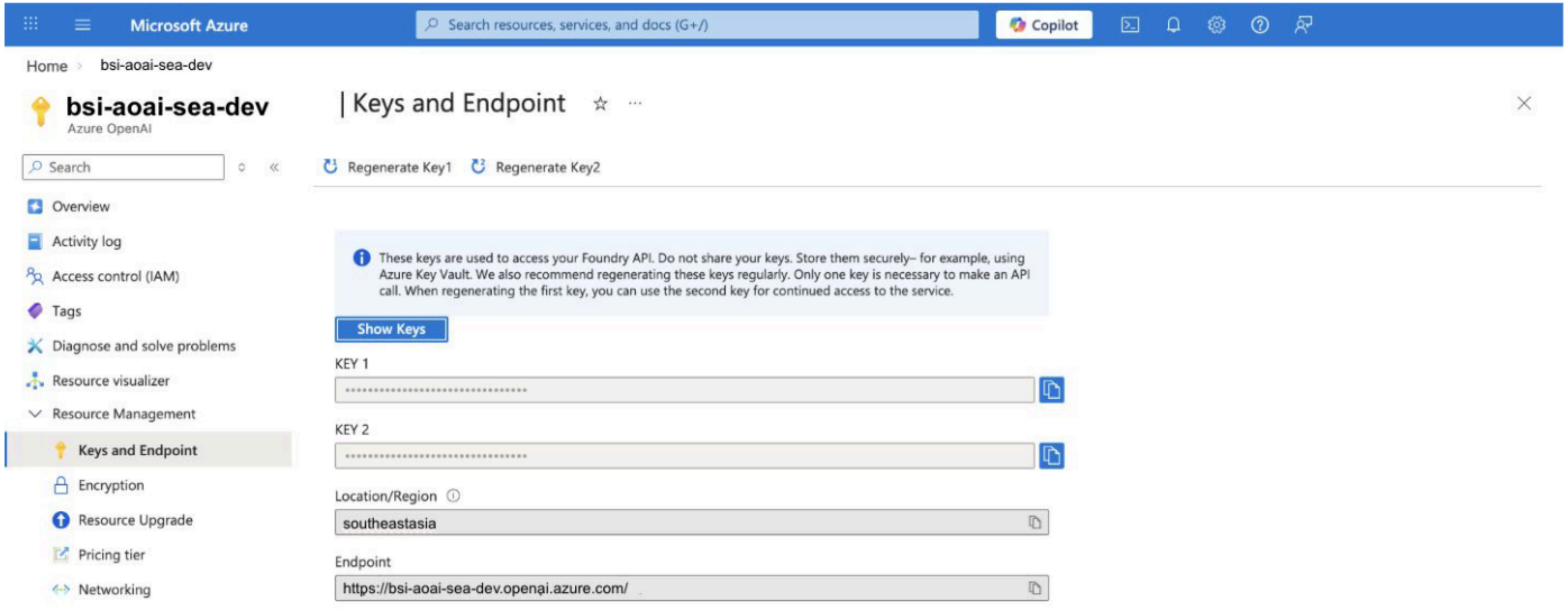
Azure OpenAI: Get the API Key

- It will open the Azure OpenAI Instance page.
- Then, click the **Endpoints** or **Manage keys**



Azure OpenAI: Get the API Key

- It will open the keys and endpoint section.
- Click the copy icon in key 1 or key 2 for the **API Key.**
- Click the copy icon in endpoint for the **Endpoint.**



Embedding Generation

Embeddings: Overview

Meaning, not surface wording

An embedding is a numeric representation of text. Humans do not read the vector directly; systems use distance and similarity between vectors to compare meaning.

Text
Original sentence or document chunk

→

Embedding vector
Semantic representation for similarity math

→

Retrieve
Nearest matches

Why retrieval gets better

Semantic closeness lets the system find related content even when the query words do not literally match the source wording.

Why this matters architecturally

Embeddings connect the API layer to the retrieval layer. Without them, knowledge retrieval is often stuck with literal term overlap.

Embeddings: Model Selection

Default

text-embedding-3-small

Practical lower-cost default for many retrieval tasks.

Sample environment output uses 1536 dimensions.

Good when cost, storage footprint, and response speed matter more than squeezing for harder retrieval quality gains.

Selective

text-embedding-3-large

Larger representation for workloads where the extra cost is justified.

Sample environment output uses 3072 dimensions.

Useful when the retrieval task is demanding enough that the bigger vector footprint earns its keep.

Selection rule

Choose based on workload quality needs, latency tolerance, storage footprint, and budget. Bigger is not automatically better.

Token Counting & Cost Estimation

Tokens and Cost Thinking

Input tokens
System prompt
user request
retrieved context

Model work
Inference runs over every token provided

Output tokens
Generated answer and completion length

Usage metadata
Input, output, and total tokens

Why tokens matter
They influence cost, latency pressure, rate usage, and how heavy each route becomes under load.

How teams estimate cost
Record usage fields, map them to the current pricing model, aggregate by route, then look for expensive patterns.

Operational habit
Reason from usage metadata first. Pricing tables change faster than the method does.

Azure OpenAI Model Pricing

To check Azure OpenAI model pricing, visit the URL: https://azure.microsoft.com/en-us/pricing/details/azure-openai

Explore pricing options

Apply filters to customize pricing options to your needs.

Prices are estimates only and are not intended as actual price quotes. Actual pricing may vary depending on the type of agreement entered with Microsoft, date of purchase, and the currency exchange rate. Prices are calculated based on US dollars and converted using London closing spot rates that are captured in the two business days prior to the last business day of the previous month end. If the two business days prior to the end of the month fall on a bank holiday in major markets, the rate setting day is generally the day immediately preceding the two business days. This rate applies to all transactions during the upcoming month. Sign in to the Azure pricing calculator to see pricing based on your current program/offer with Microsoft. Contact an Azure sales specialist for more information on pricing or to request a price quote. See frequently asked questions about Azure pricing.

Region: East US

Currency: United States – Dollar ($) USD

GPT-5.2

GPT-5.2 delivers the deep reasoning and expanded context handling necessary for building sophisticated AI agents capable of automating complex, long-running tasks across all business functions.

Model	Pricing (1M Tokens)
GPT-5.2 Codex Global	Input: $1.75 Cached Input: $0.18 Output: $14
GPT-5.2 Global	Input: $1.75 Cached Input: $0.18 Output: $14

Why Do LLMs Become Expensive

Oversized prompts

Irrelevant context raises input tokens without improving the answer.

Verbose outputs

Long completions cost more and can slow the route down.

Wrong model for the job

A more expensive model is wasteful when a smaller choice already works.

Repeated work

Recomputing embeddings or resending stable prefixes burns budget unnecessarily.

Practical response

Reduce prompt waste, right-size the model, reuse cached or stored artifacts, and watch route-level usage over time.

Processing Large Context Inputs

Large Context Risks and Mitigations

Risks to watch

More context can mean higher cost, slower requests, weaker signal-to-noise ratio, and harder reasoning about what the model actually needed.

Chunk — Break long text into units → **Retrieve** — Select only the relevant pieces → **Cache** — Reuse stable prompt prefixes → **Generate** — Answer with focused context

Mitigation principle

Give the model the best context, not the most context. Retrieval and chunking are architecture tools, not prompt-writing decoration.

Operational benefit

Focused inputs protect quality, reduce waste, and make large-context use cases easier to scale and monitor.

Prompt Caching in Practical Terms

What caching helps with

Prompt caching rewards repeated prefixes. Stable shared instructions or repeated prompt scaffolding can become cheaper and faster when the required conditions are met.

Request A

Stable prefix
+ user-specific suffix

Request B

Same prefix
+ different suffix

Remember

The repeated beginning matters most.

Use it well

Only rely on caching when the prefix is truly stable.

Do not confuse

Caching is an optimization, not a substitute for prompt discipline.

Vector Database Workflow

Vector Database Workflow

1. Prepare Collect source documents	**2. Chunk** Split into retrieval units	**3. Embed docs** Convert chunks to vectors	**4. Store** Keep text + vectors in an index	**5. Embed query** Represent the user question	**6. Retrieve** Return nearest relevant matches

Why this sequence matters

RAG quality is shaped before generation starts. Weak chunking, stale content, or poor retrieval ranking can all lower answer quality even with a strong model.

RAG Pipeline

User question

What enters the application

→

Retrieve evidence

Find the best supporting chunks

→

Ground the prompt

Attach retrieved context to the request

→

Generate

Model answers with
that evidence in view

→

Response

More relevant and traceable answer

When direct generation is enough

If the prompt already carries the needed context and the task does not depend on changing external knowledge, you may not need retrieval.

When RAG earns complexity

If the answer must be grounded in private, fresh, or controlled documents, retrieval becomes part of the system design.

Why the flow stays teachable

RAG stays understandable when it is shown as retrieval, grounding, and then generation.

Direct Generation vs RAG

Simpler

Direct generation

- Use when general knowledge or provided input is sufficient
- Best for rewriting, summarizing, formatting, and ideation
- Simpler architecture (prompt → model)
- Faster and lower cost
- ⚠ May produce outdated or hallucinated answers

Prompt → **Model**

Grounded

Retrieval Augmented Generation (RAG)

- Use when answers require external, private, or up-to-date data
- Retrieves relevant context before generating responses
- Reduces hallucination and improves accuracy
- Supports controlled and auditable knowledge sources
- Output quality depends on retrieval + generation

Query → **Retrieve context** → **Model**

Retrieval Quality Checklist

1. Chunk content so each unit keeps one coherent idea.
2. Store useful metadata so retrieval can filter intelligently.
3. Keep indexed content fresh enough for the use case.
4. Use semantic retrieval when wording mismatch is common.
5. Combine exact matching and vector similarity when identifiers matter.
6. Inspect retrieved evidence before blaming the model for weak answers.

Guided Hands-on

Token Optimization

🎯 Learning Goals

By the end of this session, participants will be able to:

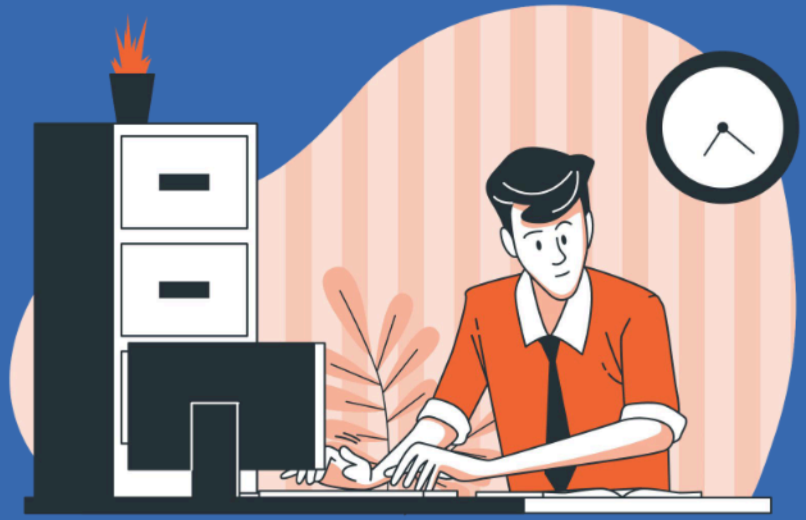
- Analyze how token usage affects LLM cost and performance.
- Optimize prompts using token estimation and reduction techniques.
- Handle long text using chunking and retrieval strategies.
- Build more efficient and scalable LLM workflows

🛠️ Setup

Participants can use:

- Google Colaboratory: https://colab.research.google.com/drive/15aDaAVcBAO6FVsYB3PpraT4L7u4JJZsl?usp=sharing

Assignment



Token Optimization with Azure OpenAI

🎯 Objective

Apply token optimization techniques using real LLM API usage and observe actual token consumption.

🧩 Scenario

You are building a Q&A assistant for company documents using Azure OpenAI.

Your challenge:

> Reduce token usage while still getting a good answer from the model.

⚙️ Requirements

Participants must:

- Use Azure OpenAI API Key
- Run code in Google Colab
- Use the provided helper functions (estimate_tokens, chunk_text, simple_retrieve)

Step 1: Setup Azure OpenAI

```
!pip install openai
```

```
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_API_KEY",
    base_url="YOUR_AZURE_ENDPOINT/openai/v1/"
)

MODEL_NAME = "YOUR_CHAT_DEPLOYMENT"
```

Step 2: Baseline (Inefficient Way)

```
all_text = " ".join([doc.content for doc in KNOWLEDGE_BASE])

response = client.responses.create(
    model=MODEL_NAME,
    input=f"Answer this question based on the context:\n{all_text}\n\nQuestion: How to handle
large prompts?"
)

print(response.output_text)
print(response.usage)
```

Task:

- Combine ALL documents
- Send to LLM
- Record token usage

👉 **Save:**

- input_tokens
- output_tokens
- total_tokens

Step 3: Optimized Approach

```
query = "How to handle large prompts?"

retrieved_docs = simple_retrieve(query, KNOWLEDGE_BASE, top_k=1)
retrieved_text = " ".join([doc.content for doc in retrieved_docs])

response_opt = client.responses.create(
    model=MODEL_NAME,
    input=f"Answer this question based on the context:\n{retrieved_text}\n\nQuestion:
{query}"
)

print(response_opt.output_text)
print(response_opt.usage)
```

Task:

- Use simple_retrieve() → top 1–2 docs only
- Send only selected content

👉 **Save:**

- input_tokens
- output_tokens
- total_tokens

Step 4: Compare Results

Answer this:

1. How many tokens were used in:
    - Baseline
    - Optimized
2. How much token reduction did you achieve?
3. Compare:
    - Response quality
    - Response speed (feel-based)

Approach	Input Tokens	Output Tokens	Total Tokens
Baseline	xxx	xxx	xxx
Optimized	xxx	xxx	xxx

HACKTIV8

Thank You

Hacktiv8id

Hacktiv8id

Hacktiv8

Hacktiv8 Indonesia